Instakill – Software Architecture Fundamentals

Week 2 Lab · C4 model and quality attribute assessment of the current monolith

Overview

Instakill is an Instagram-style social feed. Today it runs as a single Spring Boot process (Java 23) that serves both a JSON REST API, consumed by a Vue 3 single-page app, and server-rendered Thymeleaf pages. All data lives in one SQLite file. Authentication uses a JWT stored in an HttpOnly `ACCESS_TOKEN` cookie.

Inside the backend, code is organised by bounded context (`user`, `post`, `feed`, `interaction`). Each context has domain, application and API layers, and repositories are interfaces implemented by JPA adapters.

Contents

1. Context diagram (C4 level 1)
2. Container diagram (C4 level 2)
3. Component diagram of the backend (C4 level 3)
4. Quality attribute assessment

Color legend used in the diagrams

- User Instakill / Backend External system
- Frontend (Vue SPA) Database (SQLite)
- Presentation Application Domain Infrastructure
- Arrows take the color of the element (or layer) they start from.

1. Context diagram

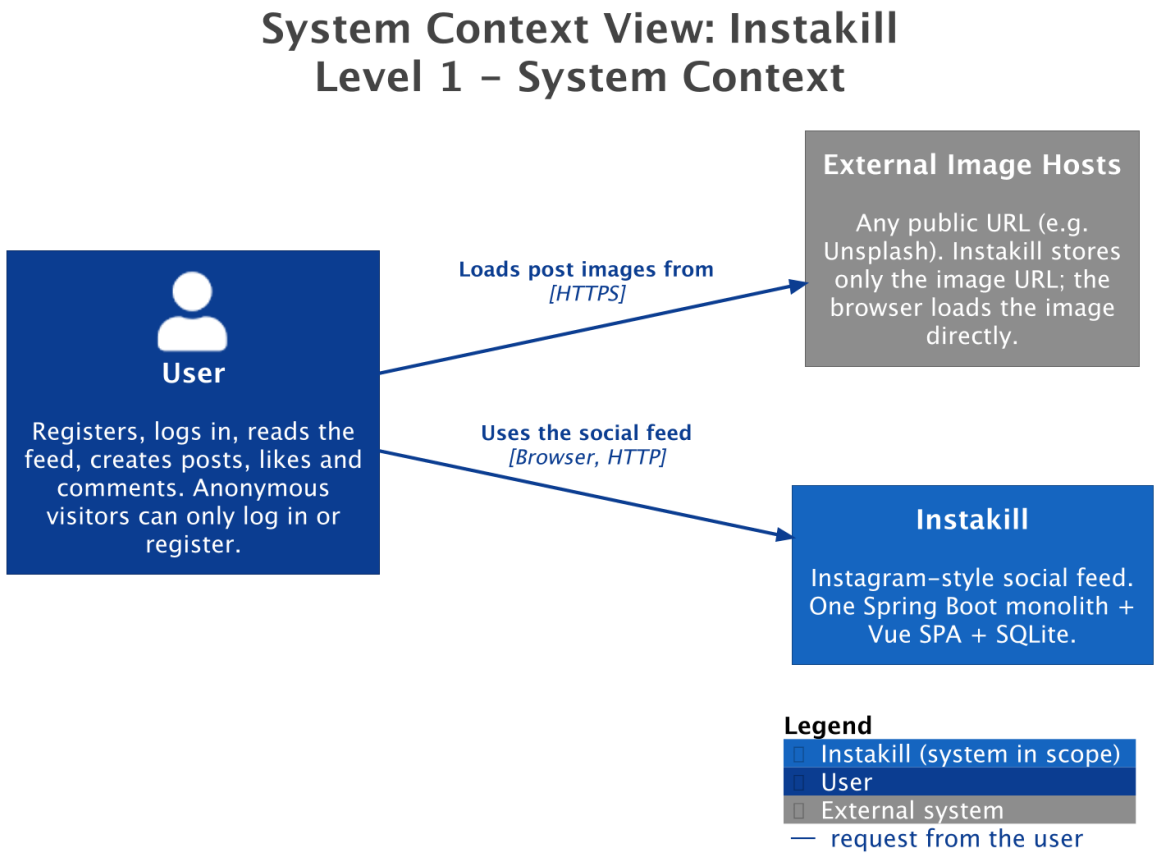


Figure 1 – Instakill in its environment. The only actor is the end user; there is no admin role. Post images are plain URLs that the browser loads directly from external hosts.

2. Container diagram

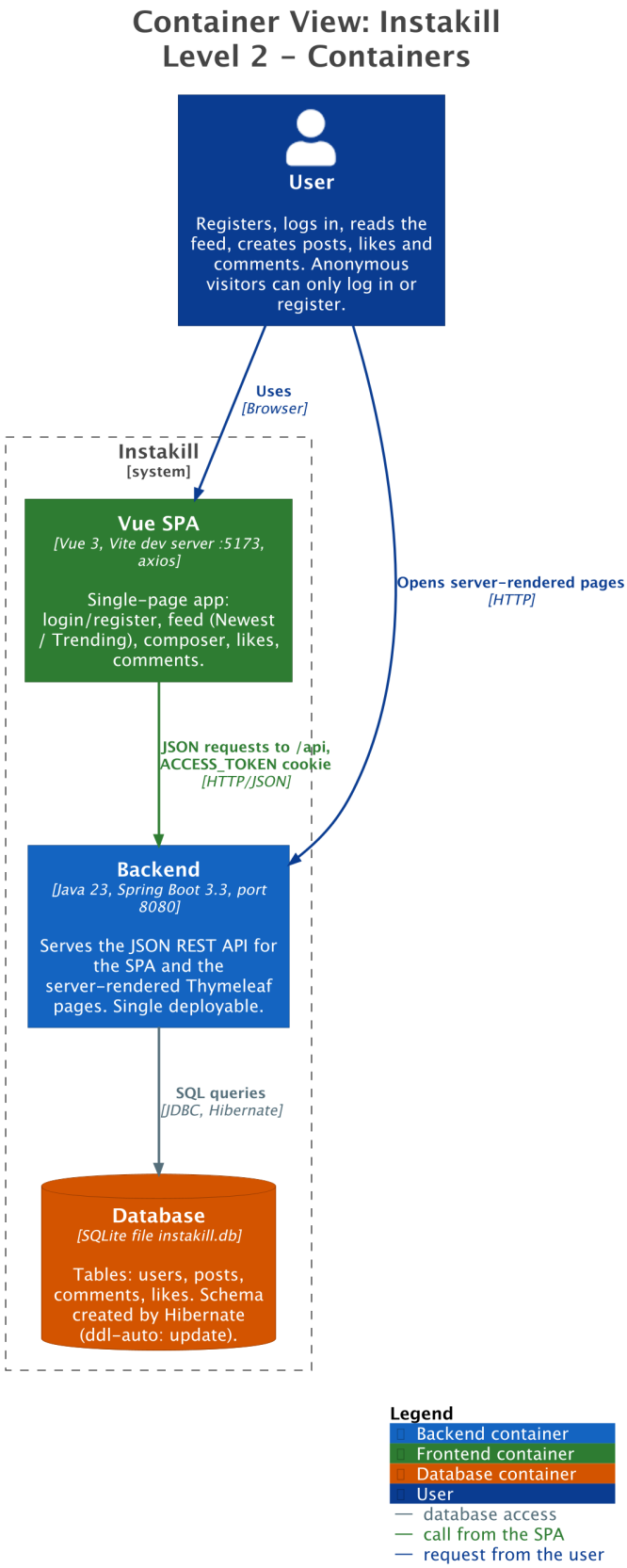


Figure 2 – The Vue SPA calls the backend's JSON API; the backend also renders HTML pages itself and talks to the SQLite database over JDBC/Hibernate.

3. Component diagram – Backend

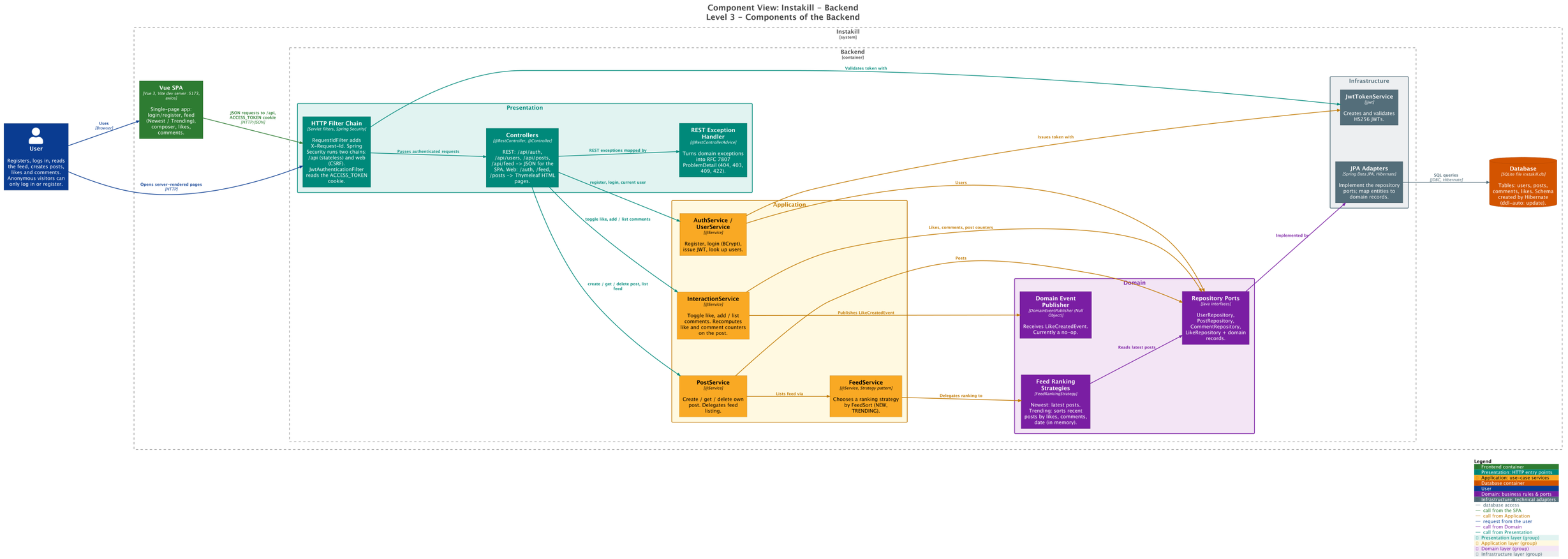


Figure 3 – Backend components grouped by layer. Requests pass the HTTP filter chain (JWT validation) into the controllers, which call application services. Services depend on repository ports in the domain, implemented by JPA adapters.

4. Quality attribute assessment

Scores go from 1 (poor coverage) to 10 (excellent coverage) and refer to the current monolithic architecture.

Attribute	Rating	Justification
Performance	**5**	Queries are paged and the hot columns are indexed (`created_at`, `post_id`, `author_id`). But the feed does one user lookup and one "liked?" lookup per post (N+1), and the Trending strategy fetches a window of recent posts and sorts it in memory. SQLite also allows only one writer at a time.
Scalability	**3**	Everything runs in one process on a local SQLite file, so the system can only scale up, not out, and the database cannot be shared by several instances. Stateless JWT authentication is the one thing that would not block running more instances.
Reliability	**4**	There is a single point of failure (one process, one DB file) with no replication or backup. Transactions and the central `RestExceptionHandler` give predictable failures, but like/comment counters are recomputed after each write with no locking, so concurrent requests can leave them stale.
Maintainability	**8**	Packages follow bounded contexts with domain / application / API layers, and repositories are interfaces implemented by JPA adapters. Strategy (feed ranking) and Null Object (event publisher) make extension cheap, and `Clock` / `IdGenerator` are injectable, which keeps tests simple. The REST and web controllers duplicate the feed-to-DTO logic, and static mappers couple `common` to the API DTOs.
Security	**6**	Passwords are hashed with BCrypt, the JWT lives in an HttpOnly `SameSite=Lax` cookie, the web chain has CSRF protection, and the API validates input. Weak points: the JWT secret is committed in `application.yaml`, the cookie is not `Secure`, the token is also returned in the login response body, there is no rate limiting or token revocation, and the generic 500 handler returns `ex.getMessage()` to clients.
Usability	**5**	The SPA covers the basic flow (register, log in, post, like, comment, sort the feed) and is simple to understand. It is minimal: no profiles, follow, or delete in the UI, no pagination controls, and two separate UIs (SPA and Thymeleaf) that behave differently.

Main gaps

1. **Scalability / reliability:** the SQLite file and in-process design are the ceiling. A server database (e.g. PostgreSQL) is the first step toward scaling out.
2. **Performance:** batch-load authors and "liked" flags for a page of posts; give Trending a real repository query (`findTrending` already exists but is unused).
3. **Security:** move the JWT secret out of the repo, set `Secure` on the cookie in production, and stop echoing internal messages on 500 errors.
4. **Maintainability:** share one feed-to-DTO assembler between the REST and web controllers.